

Tema 01

Introducción al Computador y el Lenguaje C

Programación en Lenguaje C

Ingeniería de Sistemas y Automática - Universidad de Oviedo

Sara Roos Hoefgeest Toribio



Universidad de Oviedo

Índice del Tema

1. Fundamentos del computador y ejecución secuencial de instrucciones
2. Analogía didáctica del funcionamiento interno (Páginas, PC y SW)
3. Codificación binaria, direcciones de memoria y registros
4. Estructura de la CPU: Core (UC, ALU, Reloj) y Periféricos
5. Introducción a los Lenguajes de Alto Nivel y el Compilador
6. Reglas del Lenguaje C y Entrada/Salida básica (**printf**, **scanf**)
7. Ejemplo paso a paso: Cálculo del perímetro de un polígono

El Computador y sus Instrucciones

Un computador es una máquina de ejecución secuencial de instrucciones:

- Cada instrucción es una operación muy sencilla que opera sobre datos.
Instrucciones básicas (Ensamblador):
 - **MOV** *dest*, *src* → Copia el contenido de *src* en *dest*.
 - **ADD** *dest*, *src* → Suma: $\text{dest} = \text{dest} + \text{src}$.
 - **NEG** *dest* → Cambia de signo: $\text{dest} = -\text{dest}$.
 - **DIV** *dest*, *src* → Divide: $\text{dest} = \text{dest} / \text{src}$.
- Los datos (o variables) son lugares de almacenamiento de información: guardan valores que pueden ser requeridos o modificados posteriormente.
- Un programa es una secuencia de instrucciones sencillas encargadas de resolver un problema (ej. media de 3 valores).

```
MOV m, x1    ; m = x1
ADD m, x2    ; m = m+x2 = x1+x2
ADD m, x3    ; m = m+x3 = x1+x2+x3
DIV m, 3     ; m = m/3 = (x1+x2+x3)/3
```

Codificación de Datos y Sistemas de Numeración

Un computador codifica la información en **binario** para poder almacenarla y procesarla.

Una misma cantidad puede expresarse en distintos sistemas de numeración:

- **Sistemas no ponderados** (ej. Romanos): Dificultan la realización simple de operaciones aritméticas (ej. XIV).
- **Decimal Ponderado (Base 10)**: Emplea dígitos 0..9 con pesos basados en potencias de 10 (10^1 , 10^0).
- **Binario Ponderado (Base 2)**: Emplea 0 y 1 con pesos en potencias de 2 (2^4 , 2^3 , 2^2 , 2^1 , 2^0).

Representación	Desglose de Pesos	Ejemplo de Suma Ponderada
Decimal (Base 10)	$14 = 1 \cdot 10^1 + 4 \cdot 10^0$	$14 + 27 = 41$
Romanos	$XIV = 10 - 1 + 5$	<i>Complejo de operar directamente</i>
Binario (Base 2)	$01110_2 = 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$	$01110_2 + 11011_2 = 101001_2 (41_{10})$

Organización Binaria en Memoria y Registros

Tanto los **datos** como las **instrucciones** se codifican en binario y se guardan en la memoria principal:

- **Direcciones de Memoria:** La memoria se organiza en posiciones numeradas desde 0 hasta $2^n - 1$.
- **Unidad Byte:** Cada dirección almacena exactamente 1 byte = 8 bits. Si un valor no cabe en un byte, ocupa varias direcciones consecutivas.
- **Registros de Control:**
 - **PC (Contador de Programa):** Almacena la dirección de memoria de la siguiente instrucción a ejecutar.
 - **SW (Registro de Estado):** Guarda flags de condición tras cada operación, como **ZF** (Zero Flag: se activa si el resultado es cero) y **SF** (Sign Flag: se activa si es negativo).

Organización Binaria en Memoria y Registros

PC

000..00

Dirección física de la siguiente instrucción a ejecutar

SW

Registro de Estado / Flags

ZF

SF

OF

• ZF: Cero | SF: Negativo

Dirección (RAM)	Contenido (Binario)	Interpretación / Variable
-----------------	---------------------	---------------------------

000..00

01101100

Inst1: MOV X, #3

000..01

00101110

Inst2: MOV Z, X

000..10

11111100

Inst3: ADD Z, Y

• • • (Zona de Datos / Variables) • • •

000..20

00000000

Variable X
(Valor = 0)

000..21

00000111

Variable Y
(Valor = 7)

000..22

00000010

Variable Z
(Valor = 2)

Ejecución de Programas

Un programa es una secuencia de instrucciones que el procesador ejecuta en orden secuencial:

- Cada instrucción es muy básica: usa 1, 2 o 3 operandos y realiza una copia, suma, resta, multiplicación o división.
- Los operandos se expresan como contenidos de direcciones de memoria o valores inmediatos
- Tanto instrucciones como operandos se codifican en binario

Código instruc.

01: MOV
10: ADD

Modo destino

0: Dirección
1: Inmediato

Valor destino

Dir. de X o Y

Modo fuente

0: Dirección
1: Inmediato

Valor fuente

Valor 3 o Dir. X

MOV X, #3 → Copiar a X el valor 3

01001...110100000011

ADD X, #3 → Sumar a X el valor 3

10001...110100000011

ADD Y, X → Sumar a Y el valor X

10001...111001...110

Ejecución de Programas

Para poder alterar la ejecución puramente secuencial del programa:

- Tras cada instrucción aritmética se modifica de forma automática el registro de estado (SW), indicando si el resultado fue 0 o no (ZF), negativo o no (SF), ...
- Instrucciones específicas (de salto) tienen como destino el contador de programa (PC)

Código instrucción

11: CMP
00: JMP

Modo destino / ZF

0: Dir
1: Si ZF

Valor destino / PC

Dir. X o Sumar a PC

Modo fuente

0: Dir
1: Inmed

Valor fuente

Valor (#3 o #2)

CMP X, #3 → Compara X con 3 (modifica ZF, SF)

11001...110100000011

JRELIFZ #2 → Si ZF activo ⇒ PC = PC + 2

00100000011100000010

ESQUEMA DE LA ARQUITECTURA DEL COMPUTADOR

CPU / PROCESADOR

Puede tener 1 o varios Cores conectados a la misma memoria.
Cada Core puede ejecutar secuencias de instrucciones de forma simultánea al resto, utilizando diferentes zonas de memoria.

CORE (NÚCLEO)

Unidad de Control (UC)

Decodifica y coordina instrucciones

Unidad Aritmético Lógica (ALU)

Cálculos aritméticos y lógicos

Registros: PC, SW, ...

Contienen valores temporales

Reloj: Frecuencia / CLK

MEMORIA PRINCIPAL

Instrucciones

(Código del programa)

Datos

(Variables en RAM/ROM)



BUS



E/S BUS

PERIFÉRICOS (ENTRADA / SALIDA)

Intercambian información con el "exterior"



Teclado



Pantalla



Sensores

El Computador: Periféricos

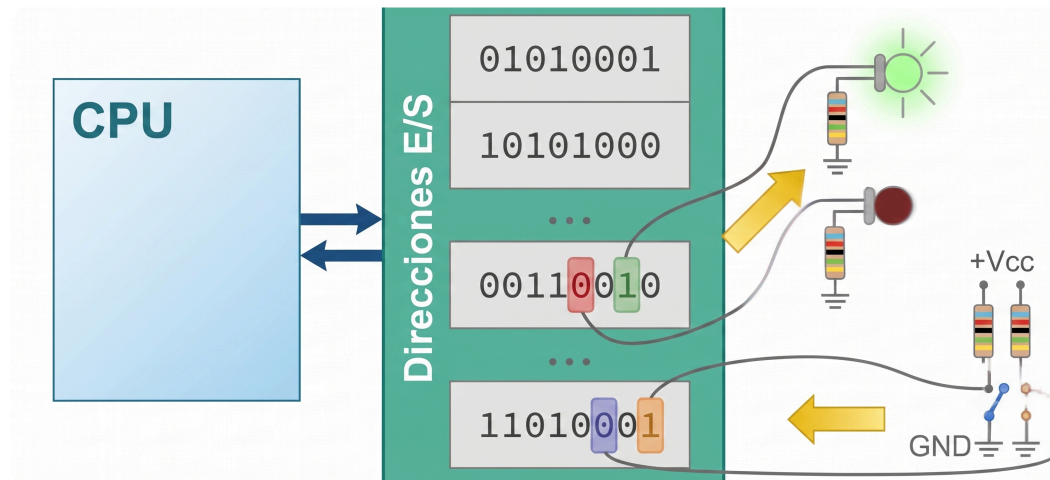
Los periféricos (o dispositivos de Entrada/Salida) sirven para que la CPU intercambie información con el “exterior”.

De cara a la CPU, la E/S es similar a la memoria:

- Está organizada en direcciones (mapa de E/S).
- Se intercambia con ella datos binarios (instrucciones IN, OUT).

La diferencia:

- Cada bit de cada dirección de E/S está físicamente conectado a un dispositivo externo.



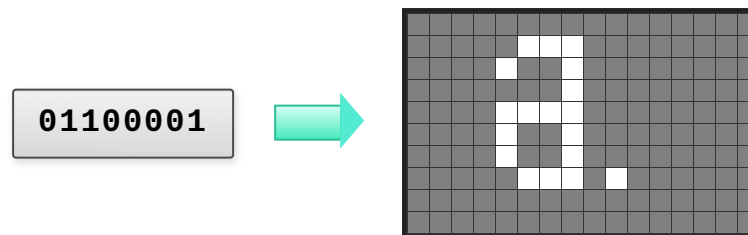
El Computador: Periféricos

La mayoría de periféricos realizan operaciones más sofisticadas con los valores binarios de las direcciones de E/S a las que están conectados:

- **Teclado:** cuando se pulsa una tecla, escribe en la dirección de E/S correspondiente un código (ASCII) según la tecla pulsada



- **Pantalla:** cuando la CPU escribe un código (ASCII) en la dirección de E/S correspondiente, genera puntos luminosos en los lugares adecuados para que el observador visualice el carácter equivalente



El Computador

¿Cómo se cargan instrucciones y datos en memoria?

- Se utiliza un programa “cargador” que vuelque los contenidos de un archivo (programa ejecutable) en memoria: instrucciones y datos

¿Cómo se ordena la ejecución del programa?

- Se indica al programa “cargador” la dirección de la 1ª instrucción.
- Una vez se ejecuta la 1ª instrucción, las siguientes se ejecutan secuencialmente (salvo los casos de salto).

¿Es necesario conocer y escribir programas en código binario?

- No, existen programas más sofisticados (compiladores e intérpretes) que nos permiten traducir a código máquina desde lenguajes de “alto nivel” (similares al lenguaje matemático).

El Lenguaje de Alto Nivel (C)

Permite escribir algoritmos utilizando notación matemática comprensible en lugar de código máquina binario.

Código Máquina / Ensamblador

```
MOV m, x1  
ADD m, x2  
ADD m, x3  
ADD m, x4  
DIV m, 4
```

Lenguaje de Alto Nivel (C)

```
int x1, x2, x3, x4;  
float m;  
  
m = (x1 + x2 + x3 + x4) / 4.0;
```

El **Compilador** es el programa encargado de traducir el código fuente a código máquina específico para cada procesador.

Reglas Fundamentales del Lenguaje C

- Declaración previa: Toda variable debe declararse con su tipo (`int`, `float`, `char`) antes de usarse.
- Sensibilidad a mayúsculas: C distingue mayúsculas de minúsculas en palabras clave e identificadores.
- Función `main()`: Todo código ejecutable debe residir dentro de una función (la principal es `main`).
- Fin de sentencia: Cada sentencia finaliza con un punto y coma (;).
- Comentarios: De línea con `//` o de bloque con `/* . . . */` (son ignorados por el compilador).

Salida por Consola

```
printf("texto a escribir", valor1, valor2, ...);
```

- Escribe en pantalla los caracteres de la cadena de texto, excepto cuando aparece `%a`lgo.
- El `%a`lgo se sustituye por el valor correspondiente de la lista (siguiendo el estricto orden de aparición):
 - `%d`: entero (`int`)
 - `%f`: real / flotante (`float`)
 - `%c`: carácter individual (`char`)
 - `%s`: cadena de caracteres (texto)
- La secuencia de escape `\n` en el texto representa un salto de línea (retorno de carro).

Ejemplo de Código C:

```
int alumnos = 12;  
float nota_media = 7.25;  
printf("Hay %d alumnos, nota media %f\n", alumnos, nota_media);
```

Resultado en pantalla:

```
Hay 12 alumnos,  
nota media  
7.250000
```



Entrada por Consola

```
scanf("formato", direccion_valor1, direccion_valor2, ...);
```

- Espera por teclado la pulsación de una secuencia de caracteres terminada en **INTRO**.
- El texto del formato **sólo** debe contener cadenas de formato (%a lgo).
- Las direcciones se indican anteponiendo a cada variable el carácter &.
- El usuario debe separar cada entrada con **espacio**, **tabulador** o **Return**, y siempre terminar con Return.

Ejemplo de Código C:

```
int alumnos;  
float nota_media;  
  
printf("Introduzca alumnos y nota media: ");  
scanf("%d %f", &alumnos, &nota_media);  
printf("Hay %d alumnos, nota media %f\n", alumnos, nota_media);
```

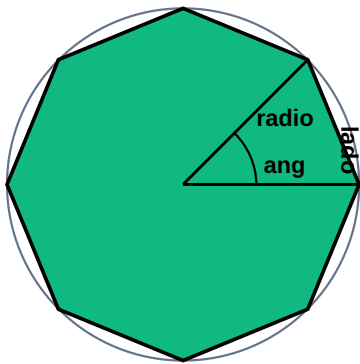
Resultado en pantalla:

```
Introduzca  
alumnos y nota  
media: 12 7.25↵  
Hay 12 alumnos,  
nota media  
7.250000  
█
```

 **Nota:** Olvidar el operador & en scanf() provocará un error de ejecución al intentar escribir en una dirección de memoria errónea.

El lenguaje de alto nivel (C)

Ejemplo: dados el radio y el número de lados de un polígono regular, calcular su perímetro



Datos:

n_lados (debe ser entero)

radio (puede ser real)

Resultado:

perim (puede ser real)

Cálculo:

$$\textit{perim} = \textit{lado} * \textit{n_lados}$$

donde ***lado*** se obtiene de:

$$\sin(\textit{ang}/2) = \frac{\textit{lado}/2}{\textit{radio}}$$

y ***ang*** se obtiene de:

$$\textit{ang} = 2\pi / \textit{n_lados}$$

En ***rojo/cursiva***: variables necesarias (cambian su valor en función de los datos iniciales y/o de la evolución de los cálculos).

Ejemplo Completo: Perímetro de un Polígono

Implementación en C del cálculo del perímetro de un polígono regular

```
#include <stdio.h>
#include <math.h>

#define PI 3.1416 // Constante

int main(void) {
    int n_lados;      // Entrada: número de lados
    float radio;      // Entrada: radio del polígono
    float perim;      // Salida: perímetro calculado
    float ang, lado;  // Variables auxiliares

    printf("Introduzca n lados: ");
    scanf("%d", &n_lados);
    printf("Introduzca radio: ");
    scanf("%f", &radio);

    ang = 2 * PI / n_lados;
    lado = 2 * radio * sin(ang / 2.0f);
    perim = n_lados * lado;

    printf("El perímetro es %f\n", perim);
}
```

Ejecución, paso por paso (1/8): Declaración de variables

Código fuente:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #define PI 3.1416 // Constante
5
6 main()
7 {
8     int n_lados; // Dato de usuario
9     float radio; // Dato de usuario
10    float perim; /* Resultado */
11    float ang,lado; // Valores aux
12    printf("Introduzca n lados: ");
13    scanf("%d",&n_lados);
14    printf("Introduzca radio: ");
15    scanf("%f",&radio);
16
17    ang=2*PI/n_lados;
18    lado=2*radio*sin(ang/2);
19    perim=n_lados*lado;
20
21    printf("El perim es %f\n",perim);
22 }
```

Acción:

Reserva lugares en memoria para las variables declaradas y "recuerda" la posición y tipo de cada una.



Memoria RAM:

8000	a	8003	?	n_lados
8004	a	8007	?	radio
8008	a	8011	?	perim
8012	a	8015	?	ang
8016	a	8019	?	lado

Consola (pantalla + teclado):



Ejecución, paso por paso (2/8): Primer printf()

Código fuente:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #define PI 3.1416 // Constante
5
6 main()
7 {
8     int n_lados; // Dato de usuario
9     float radio; // Dato de usuario
10    float perim; /* Resultado */
11    float ang,lado; // Valores aux
12
13    printf("Introduzca n lados: ");
14    scanf("%d",&n_lados);
15    printf("Introduzca radio: ");
16    scanf("%f",&radio);
17
18    ang=2*PI/n_lados;
19    lado=2*radio*sin(ang/2);
20    perim=n_lados*lado;
21
22    printf("El perim es %f\n",perim);
23 }
```

Acción:

Invoca a la función `printf()` que escribe el texto indicado en pantalla.



Memoria RAM:

8000 a 8003	?	n_lados
8004 a 8007	?	radio
8008 a 8011	?	perim
8012 a 8015	?	ang
8016 a 8019	?	lado

Consola (pantalla + teclado):

Introduzca n lados: █

Ejecución, paso por paso (3/8): Primer scanf()

Código fuente:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #define PI 3.1416 // Constante
5
6 main()
7 {
8     int n_lados; // Dato de usuario
9     float radio; // Dato de usuario
10    float perim; /* Resultado */
11    float ang,lado; // Valores aux
12
13    printf("Introduzca n lados: ");
14    scanf("%d",&n_lados);
15    printf("Introduzca radio: ");
16    scanf("%f",&radio);
17
18    ang=2*PI/n_lados;
19    lado=2*radio*sin(ang/2);
20    perim=n_lados*lado;
21
22    printf("El perim es %f\n",perim);
23 }
```

Acción:

Invoca a `scanf ( )` que espera pulsaciones por teclado terminadas en INTRO y asigna el valor entero a `n_lados` (`&n_lados = 8000`).



Memoria RAM:

8000	a	8003	8	n_lados
8004	a	8007	?	radio
8008	a	8011	?	perim
8012	a	8015	?	ang
8016	a	8019	?	lado

Consola (pantalla + teclado):

```
Introduzca n lados: 8 ↵
█
```

Ejecución, paso por paso (4/8): Lectura del radio

Código fuente:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #define PI 3.1416 // Constante
5
6 main()
7 {
8     int n_lados; // Dato de usuario
9     float radio; // Dato de usuario
10    float perim; /* Resultado */
11    float ang,lado; // Valores aux
12
13    printf("Introduzca n lados: ");
14    scanf("%d",&n_lados);
15    printf("Introduzca radio: ");
16    scanf("%f",&radio);
17
18    ang=2*PI/n_lados;
19    lado=2*radio*sin(ang/2);
20    perim=n_lados*lado;
21
22    printf("El perim es %f\n",perim);
23 }
```

Acción:

Lectura por teclado del dato para radio (%f para float, dirección &radio = 8004).



Memoria RAM:

8000	a	8003	8	n_lados
8004	a	8007	12.5	radio
8008	a	8011	?	perim
8012	a	8015	?	ang
8016	a	8019	?	lado

Consola (pantalla + teclado):

```
Introduzca n lados: 8 ↵
Introduzca radio: 12.5 ↵
█
```

Ejecución, paso por paso (5/8): Cálculo del ángulo

Código fuente:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #define PI 3.1416 // Constante
5
6 main()
7 {
8     int n_lados; // Dato de usuario
9     float radio; // Dato de usuario
10    float perim; /* Resultado */
11    float ang,lado; // Valores aux
12
13    printf("Introduzca n lados: ");
14    scanf("%d",&n_lados);
15    printf("Introduzca radio: ");
16    scanf("%f",&radio);
17
18    ang=2*PI/n_lados;
19    lado=2*radio*sin(ang/2);
20    perim=n_lados*lado;
21
22    printf("El perim es %f\n",perim);
23 }
```

Acción:

Ejecuta los cálculos
secuencialmente y
asigna el resultado a la
variable ang.



Memoria RAM:

8000 a	8	n_lados
8003		
8004 a	12.5	radio
8007		
8008 a	?	perim
8011		
8012 a	0.78540	ang
8015		
8016 a	?	lado
8019		

Consola (pantalla + teclado):

```
Introduzca n lados: 8 ↵
Introduzca radio: 12.5 ↵
```

Ejecución, paso por paso (6/8): Cálculo del lado

Código fuente:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #define PI 3.1416 // Constante
5
6 main()
7 {
8     int n_lados; // Dato de usuario
9     float radio; // Dato de usuario
10    float perim; /* Resultado */
11    float ang,lado; // Valores aux
12
13    printf("Introduzca n lados: ");
14    scanf("%d",&n_lados);
15    printf("Introduzca radio: ");
16    scanf("%f",&radio);
17
18    ang=2*PI/n_lados;
19    lado=2*radio*sin(ang/2);
20    perim=n_lados*lado;
21
22    printf("El perim es %f\n",perim);
23 }
```

Acción:

Calcula la longitud de uno de los lados usando la función `sin()` de `math.h` y guarda el valor en `lado`.



Memoria RAM:

8000 a	8	n_lados
8003		
8004 a	12.5	radio
8007		
8008 a	?	perim
8011		
8012 a	0.78540	ang
8015		
8016 a	9.5918	lado
8019		

Consola (pantalla + teclado):

```
Introduzca n lados: 8 ↵
Introduzca radio: 12.5 ↵
█
```

Ejecución, paso por paso (7/8): Cálculo del perímetro

Código fuente:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #define PI 3.1416 // Constante
5
6 main()
7 {
8     int n_lados; // Dato de usuario
9     float radio; // Dato de usuario
10    float perim; /* Resultado */
11    float ang,lado; // Valores aux
12
13    printf("Introduzca n lados: ");
14    scanf("%d",&n_lados);
15    printf("Introduzca radio: ");
16    scanf("%f",&radio);
17
18    ang=2*PI/n_lados;
19    lado=2*radio*sin(ang/2);
20    perim=n_lados*lado;
21
22    printf("El perim es %f\n",perim);
23 }
```

Acción:

Multiplica el número de lados por la longitud de cada lado y guarda el resultado final en perim.



Memoria RAM:

8000 a	8	n_lados
8003		
8004 a	12.5	radio
8007		
8008 a	57.551	perim
8011		
8012 a	0.78540	ang
8015		
8016 a	9.5918	lado
8019		

Consola (pantalla + teclado):

```
Introduzca n lados: 8 ↵
Introduzca radio: 12.5 ↵
█
```

El lenguaje de alto nivel (C)

Ejecución, paso por paso (8/8): Muestra de resultado final

Código fuente:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #define PI 3.1416 // Constante
5
6 main()
7 {
8     int n_lados; // Dato de usuario
9     float radio; // Dato de usuario
10    float perim; /* Resultado */
11    float ang,lado; // Valores aux
12
13    printf("Introduzca n lados: ");
14    scanf("%d",&n_lados);
15    printf("Introduzca radio: ");
16    scanf("%f",&radio);
17
18    ang=2*PI/n_lados;
19    lado=2*radio*sin(ang/2);
20    perim=n_lados*lado;
21
22    printf("El perim es %f\n",perim);
23 }
```

Acción:

Invoca a `printf()` que sustituye el marcador `%f` por el valor almacenado en `perim` y lo escribe en la pantalla.



Memoria RAM:

8000 a	8	n_lados
8003		
8004 a	12.5	radio
8007		
8008 a	57.551	perim
8011		
8012 a	0.78540	ang
8015		
8016 a	9.5918	lado
8019		

Consola (pantalla + teclado):

```
Introduzca n lados: 8 ↵
Introduzca radio: 12.5 ↵
El perim es 57.551000
█
```

Ejercicios propuestos

1. Pedir dos conjuntos de hora, min, seg y calcular el tiempo en segundos entre ambos.
2. Realizar un programa que pida el tamaño de los 2 lados de un rectángulo, y calcule y escriba su diagonal.
3. Pedir dos puntos de una recta en el espacio bidimensional, calcular los parámetros de la recta $y = mx + n$ que los une, y escribir el valor de la ordenada y para cualquier valor de la abscisa x .
4. Pedir una temperatura en °F y calcular su equivalente en °C
(https://es.m.wikipedia.org/wiki/Archivo:Fahrenheit_Celsius_scales.svg)
5. Pedir el valor de velocidad (Km/h) en dos instantes, el tiempo (min) entre ambos, y calcular: el espacio recorrido (en m) y la aceleración (en m/s^2)

NOTAS:

Algunas operaciones matemáticas en C (incluir `<math.h>`):

$$x^2 = x * x \quad \sqrt{x} = \text{sqrt}(x)$$

Operaciones trigonométricas: $\sin(x)$, $\cos(x)$, $\tan(x)$, $\text{atan}(x)$

División entera. Cociente = a/b ; Resto = $a\%b$

Resumen del Tema

1. Un computador ejecuta instrucciones binarias simples de manera secuencial controladas por el registro PC.
2. La memoria se organiza en direcciones de 1 byte, almacenando tanto código como variables.
3. Los lenguajes de alto nivel como C permiten expresar algoritmos matemáticos legibles que el compilador traduce a binario.
4. **printf** y **scanf** gestionan la comunicación con la consola usando especificadores de formato (%d, %f).